

Препоръчваща система за книги

Документация и резултати от offline оценка

Курсов проект — Препоръчващи системи



Никол Николаева, Габриела Костева

Hybrid препоръчване с content-based filtering, collaborative filtering, popularity, social signal и MMR. Оценка: leave-last-out, Precision@K, Recall@K, NDCG@K.

Съдържание

1. Въведение
2. Функционалности на системата
3. Архитектура и източници на данни
4. Алгоритми
5. Методи за оценяване
6. Експеримент: еволюция на данните
7. Резултати от оценката
8. Графики
9. Извод
10. Стартиране и възпроизвеждане

1. Въведение

В ерата на дигиталните библиотеки и онлайн книжарници читателите имат достъп до десетки хиляди заглавия, но все по-често се сблъскват с един и същ практически проблем: трудно е да открият книга, която наистина отговаря на личните им интереси. Класическите класации по жанр или автор са полезни, но не улавят нюансите на индивидуалния вкус — комбинация от предпочитани теми, език, настроение и предишен опит с конкретни книги.

Настоящият проект реализира препоръчваща система за книги — веб приложение с REST API, което предлага персонализирани Top-N препоръки. Системата е разработена в рамките на курса по препоръчващи системи във ФМИ и демонстрира пълен цикъл: дефиниране на проблем, избор на алгоритми, реализация, демонстрационен интерфейс и offline оценка на качеството.

1.1 Проблем и мотивация

Основният фокус е върху cold start — ситуацията, в която нов потребител все още няма достатъчна история от харесвания, оценки или прочетени книги. В този режим чистият collaborative filtering (CF) често не работи, защото липсват данни за сходни потребители. Затова системата комбинира няколко подхода:

- content-based filtering (CBF) — анализ на текст и метаданни на книгите;
- collaborative filtering (CF) — моделиране на потребителско поведение чрез user-item матрица;
- popularity — baseline по глобално търсени заглавия;
- social signal — влияние на приятели в платформата.

Тези компоненти се обединяват в hybrid модел с претеглено подреждане и MMR (Maximal Marginal Relevance) за по-голямо разнообразие в препоръките. Всеки резултат се придружава от кратко текстово обяснение (жанр, автор, подобна книга, приятел и др.).

1.2 Цели на проекта

1. Да се изгради работещо демо с регистрация, каталог, лична библиотека и персонализиран feed.
2. Да се реализира hybrid препоръчване с поддръжка на cold start (анкета при регистрация и бърз implicit feedback).
3. Да се интегрира social signal при взаимни приятелства.
4. Да се проведе offline оценка с leave-last-out и метрики Precision@K, Recall@K, NDCG@K.
5. Да се документира еволюцията на системата чрез контролирани подобрения върху данните.

2. Функционалности на системата

2.1 Потребителски акаунти и сигурност

- Регистрация и вход с парола, криптирана чрез bcrypt.
- Сесия чрез HTTP cookie (recsys_session).
- Кратка анкета при запис (темпо на четене, настройване, формат) — допълнителен сигнал за CBF.

2.2 Cold start

След регистрация или вход без история се показва модален мини-quiz:

- избор на жанрове, език и любими автори;
- бърз implicit feedback към книги от каталога (Like / Dislike / «Чел(а) съм»).

Данните се записват в профила и в таблицата с взаимодействия.

2.3 Каталог и данни

- Начален каталог от data/books.csv (UTF-8, разделител ;).
- Импорт на нови книги от Open Library API (без API ключ) — от UI или CLI.
- Автоматични корици от Open Library по ISBN; резервен placeholder при липса на изображение.

2.4 Препоръки и интерфейс

Таб / функция	Описание
---------------	----------

За теб	Персонализирани Top-N препоръки от hybrid модела
Каталог	Търсене, преглед, импорт от мрежата
Популярни	Подреждане по глобални положителни сигнали
Библиотека	Лична история: харесвания, оценки, прочетени, за четене
Приятели	Добавяне, заявки, взаимни връзки
Оценка	Offline метрики и методологически текст

2.5 Обяснения и разнообразие

Всяка препоръка включва кратко обяснение: съвпадение по жанр/автор, подобна вече харесана книга, данни от анкетата или social signal («Приятел X хареса тази книга»). След първоначалното подреждане се прилага MMR за по-разнообразен Top-K списък.

3. Архитектура и източници на данни

3.1 Технологичен стек

- Backend: Python 3.11+, FastAPI, SQLAlchemy
- База данни: SQLite (data/recsys.db)
- ML: scikit-learn (TF-IDF, Truncated SVD), NumPy, SciPy
- Frontend: Jinja2 + JavaScript (единична страница с табове)

3.2 Поток на данните

Книгите не се четат директно от CSV при всяка заявка. CSV и Open Library попълват локалната база; препоръките се изчисляват от SQLite. Потребителските действия (like, dislike, rating, finished, to_read) се натрупват в реално време и обновяват модела.

3.3 Източници на книги

- data/books.csv — стартов каталог (~154 книги)
- Open Library API — динамично разширяване (search.json, covers)
- UI / CLI импорт — upsert по ISBN или заглавие+автор

4. Алгоритми

4.1 Content-Based Filtering (CBF)

Върху всяка книга се изгражда текстов документ от заглавие, автор, описание и тагове. Използва се TF-IDF (до 6000 features, n-grams 1–2). Профилът на потребителя е претеглена средна стойност на векторите на книгите, с които е взаимодействал, плюс onboarding предпочитания и анкета.

4.2 Collaborative Filtering (CF)

Построява се разредена user-item матрица с implicit тегла (like, finished, rating ≥ 4). Ниските оценки не се третират като положителен сигнал. Матрицата се факторизира чрез Truncated SVD (12 компоненти). Предсказанията идват от скалярно произведение на user и item фактори.

4.3 Popularity baseline

Глобален вектор по брой положителни взаимодействия (like, finished, rating ≥ 4), нормализиран. Служи като baseline и като компонент в hybrid модела.

4.4 Social signal

При взаимни приятелства се агрегират харесвания, прочетени книги и високи оценки на приятелите в отделен вектор, нормализиран по максимума.

4.5 Hybrid модел

Теглата зависят от броя значими взаимодействия на потребителя:

- Cold start (≤ 6 взаимодействия): ~55% CBF, ~15% CF, до ~15% social, остатък popularity.
- Top users: 45% CF, 35% CBF, 20% social (ако има приятели), остатък popularity.

4.6 MMR (Maximal Marginal Relevance)

От първите ~80 кандидата по релевантност се избират K книги с баланс между score и различие спрямо вече избраните ($\lambda = 0.7$ по подразбиране).

5. Методи за оценяване

5.1 Протокол: leave-last-out

1. За всеки потребител с поне един положителен сигнал (like, rating ≥ 4 , finished) се скрива последният такъв запис по време.
2. CF се обучава без този запис; CBF и hybrid не виждат същия ред за съответния потребител.
3. Всички книги се подреждат; проверява се дали скритата книга е в Top-K.
4. Метриците се усредняват по потребителите във fold-a.

5.2 Метрики

- Precision@K — колко от Top-K са релевантни (тук: една скрита книга).
- Recall@K — дял от релевантните, открити в Top-K.
- NDCG@K — качество на позицията в списъка (по-високо = по-добре).

5.3 Сравнявани методи

1. Popular — само popularity
2. CBF-only — само content-based filtering
3. CF-only — само collaborative filtering
4. Hybrid — комбиниран модел
5. Hybrid+Social — hybrid с усилен social signal

5.4 Общ резултат (индекс O)

За обобщаваща графика: $O = (\text{Precision@K} + \text{Recall@K} + \text{NDCG@K}) / 3$

6. Експерименти и оптимизации

6.1 Версия 0 — базова линия

Демо seed: 8 потребителя, 154 книги, 144 взаимодействия. CF и Hybrid дават нулеви метрики — типично при оскъдна user-item матрица.

Показател	Стойност
Книги	154
Потребители	8
Взаимодействия	144
Приятелства	14
Fold	8

6.2 Оптимизация 2 — подобрения върху каталога (v1)

- +52 книги от Open Library (historical fiction, science fiction, mystery)
- Обогащаване на тагове за 135 книги (правило-базирано)
- +16 приятелства между демо акаунтите

Показател	Стойност
Книги	206
Потребители	8
Взаимодействия	144
Приятелства	30

Най-добър общ резултат (O): Popular = 0.2345; останалите методи = 0.

6.3 Експеримент 1а — синтетични данни, вълна 1 (v2)

35 нови потребителя (synth_01 ... synth_35) с 14–18 взаимодействия всеки.

Разпределението имитира демо seed и API потока:

- rating с 3★ / 4★ / 5★ (тежест към по-високи оценки при висока affinity)
- like, finished, to_read
- рядко dislike при ниска affinity

Промяна	Резултат
Нови потребители	35
Взаимодействия	+549
Приятелства	+80
Показател	Стойност
Книги	206

Показател	Стойност			
Потребители	43			
Взаимодействия	693			
Приятелства	110			
Fold	43			
Метод	P@10	R@10	NDCG@10	O
Popular	0.0209	0.2093	0.1036	0.1113
CBF-only	0.0116	0.1163	0.0612	0.0630
CF-only	0.0116	0.1163	0.0400	0.0560
Hybrid	0.0093	0.0930	0.0348	0.0457
Hybrid+Social	0.0116	0.1163	0.0441	0.0573

6.4 Експеримент 1b — синтетични данни, вълна 2 (v3)

- +8 нови взаимодействия на всеки съществуващ synth потребител (без дублиране user-book)
- +20 нови потребителя (synth_36 ... synth_55) с 12–16 взаимодействия
- Допълнителни приятелства между демо и synth мрежата

Цел: по-плътна user-item матрица и по-стабилен CF / Hybrid offline резултат.

Промяна	Резултат
Допълнителни взаимодействия	+565
Нови потребители	20
Приятелства	+54

Показател	Стойност			
Книги	206			
Потребители	63			
Взаимодействия	1258			
Приятелства	164			
Fold	63			
Метод	P@10	R@10	NDCG@10	O
Popular	0.0143	0.1429	0.0545	0.0706
CBF-only	0.0190	0.1905	0.0741	0.0945
CF-only	0.0063	0.0635	0.0244	0.0314

Метод	P@10	R@10	NDCG@10	O
Hybrid	0.0095	0.0952	0.0419	0.0489
Hybrid+Social	0.0143	0.1429	0.0649	0.0740

6.5 Оптимизация 3 — hyperparameter tuning (v4)

Grid search върху същия leave-last-out fold (K=10), 135 комбинации:

Стъпка А — тегла без MMR: weight_cf, weight_cbf, weight_social, cold-start тройки, social_blend_hybrid / social_blend_social.

Стъпка В — при най-добрите тегла: търсене на MMR $\lambda \in \{0.60 \dots 0.90\}$ с production-like candidate pool.

Резултатът се записва в data/tuned_hybrid.json. Скрипт: python scripts/tune_hybrid.py.

Параметър	Стойност
weight_cf / weight_cbf / weight_social	0.35 / 0.35 / 0.25
cold_weight_cf / cbf / social	0.15 / 0.55 / 0.15
social_blend_hybrid / social	0.65 / 0.35
MMR λ	0.7

Метрика (tuning objective)	Преди	След
Combined NDCG (Hybrid + Hybrid+Social)	0.0534	0.0579

Метрики @ K=10 (същите данни като v3):

Метод	P@10	R@10	NDCG@10	O	Δ NDCG vs v3
Popular	0.0143	0.1429	0.0545	0.0706	—
CBF-only	0.0190	0.1905	0.0741	0.0945	—
CF-only	0.0063	0.0635	0.0244	0.0314	—
Hybrid	0.0095	0.0952	0.0469	0.0505	+0.0050
Hybrid+Social	0.0143	0.1429	0.0689	0.0754	+0.0040

Забележка

Синтетичните данни служат за offline експеримент; не заменят реални потребителски тестове или A/B експерименти.

Обобщение на еволюцията

- v0 → v1: по-богат каталог; Popular се подобрява силно
- v1 → v2 → v3: повече потребители и ratings; CF и Hybrid стават измерими
- v3 → v4: tuning подобрява Hybrid+Social NDCG@10 от 0.0649 → 0.0689 без нови данни

7. Резултати от оценката

Оценката е offline с протокол **leave-last-out** при **K = 10**. За всеки потребител с положителен сигнал (like, rating ≥ 4 , finished) се скрива последният такъв запис; моделите се оценяват без него. Усредняват се **Precision@10**, **Recall@10** и **NDCG@10** по потребителите във fold-a.

Общ индекс: $O = (\text{Precision@K} + \text{Recall@K} + \text{NDCG@K}) / 3$

Сравнявани методи: **Popular**, **CBF-only**, **CF-only**, **Hybrid**, **Hybrid+Social**.

Таблица 1 — Базова линия (v0)

Книги: 154 | Потребители: 8 | Взаимодействия: 144 | Fold: 8

Метод	P@10	R@10	NDCG@10	O
Popular	0.0125	0.1250	0.0394	0.0590
CBF-only	0.0125	0.1250	0.0376	0.0584
CF-only	0.0000	0.0000	0.0000	0.0000
Hybrid	0.0000	0.0000	0.0000	0.0000
Hybrid+Social	0.0000	0.0000	0.0000	0.0000

Таблица 2 — След подобрения на каталога (v1)

Книги: 206 | Потребители: 8 | Взаимодействия: 144 | Fold: 8

Метод	P@10	R@10	NDCG@10	O
Popular	0.0500	0.5000	0.1534	0.2345
CBF-only	0.0000	0.0000	0.0000	0.0000
CF-only	0.0000	0.0000	0.0000	0.0000
Hybrid	0.0000	0.0000	0.0000	0.0000
Hybrid+Social	0.0000	0.0000	0.0000	0.0000

Таблица 3 — Синтетични данни, вълна 1 (v2)

Книги: 206 | Потребители: 43 | Взаимодействия: 693 | Fold: 43

Метод	P@10	R@10	NDCG@10	O
Popular	0.0209	0.2093	0.1036	0.1113
CBF-only	0.0116	0.1163	0.0612	0.0630
CF-only	0.0116	0.1163	0.0400	0.0560
Hybrid	0.0093	0.0930	0.0348	0.0457
Hybrid+Social	0.0116	0.1163	0.0441	0.0573

Таблица 4 — Синтетични данни, вълна 2 (v3)

Книги: 206 | Потребители: 63 | Взаимодействия: 1258 | Fold: 63

Метод	P@10	R@10	NDCG@10	O
Popular	0.0143	0.1429	0.0545	0.0706
CBF-only	0.0190	0.1905	0.0741	0.0945
CF-only	0.0063	0.0635	0.0244	0.0314
Hybrid	0.0095	0.0952	0.0419	0.0489
Hybrid+Social	0.0143	0.1429	0.0649	0.0740

Таблица 5 — След hyperparameter tuning (v4, финал)

Същите данни като v3; приложени тегла от data/tuned_hybrid.json.

Книги: 206 | Потребители: 63 | Взаимодействия: 1258 | Fold: 63

Метод	P@10	R@10	NDCG@10	O	Δ NDCG vs v3
Popular	0.0143	0.1429	0.0545	0.0706	—
CBF-only	0.0190	0.1905	0.0741	0.0945	—
CF-only	0.0063	0.0635	0.0244	0.0314	—
Hybrid	0.0095	0.0952	0.0469	0.0505	+0.0050
Hybrid+Social	0.0143	0.1429	0.0689	0.0754	+0.0040

Сравнение база (v0) → финал (v4)

Метод	NDCG@10 (v0)	NDCG@10 (v4)	O (v0)	O (v4)
Popular	0.0394	0.0545	0.0590	0.0706
CBF-only	0.0376	0.0741	0.0584	0.0945
CF-only	0.0000	0.0244	0.0000	0.0314
Hybrid	0.0000	0.0469	0.0000	0.0505
Hybrid+Social	0.0000	0.0689	0.0000	0.0754

Кратки изводи от резултатите

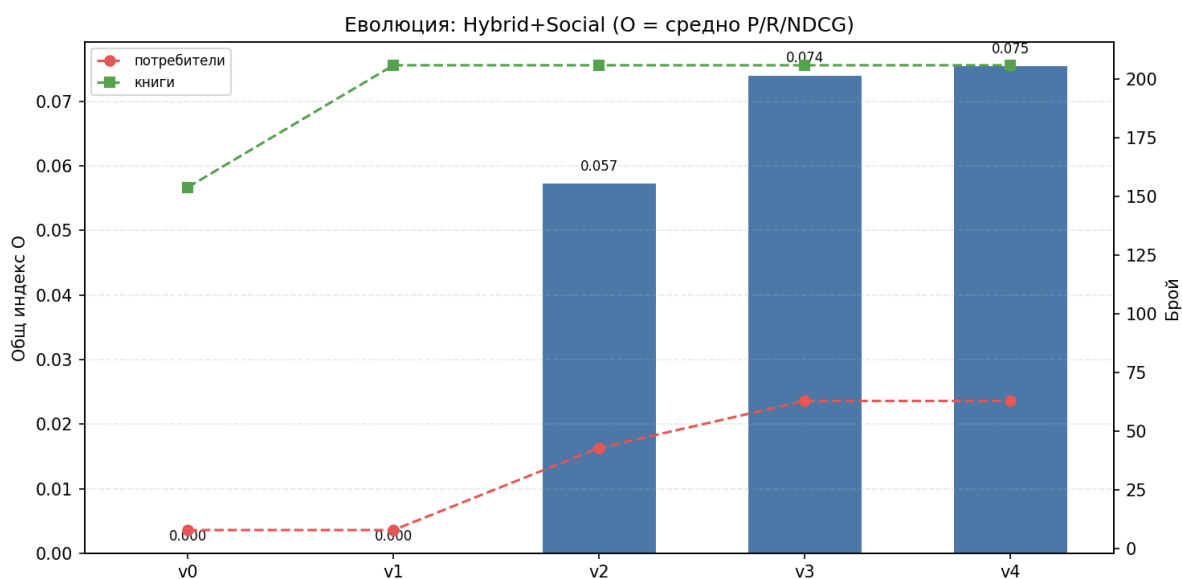
1. При 8 потребителя (v0) само **Popular** и **CBF-only** имат ненулеви метрики; **CF** и **Hybrid** не се обучават.
2. След разширяване на каталога без нови потребители (v1) **Popular** скача рязко (NDCG@10 = 0.1534).
3. Синтетичните данни (v2, v3) активират **CF**, **Hybrid** и **Hybrid+Social**.
4. При финалния обем данни (v3/v4) най-висок **O** дава **CBF-only** (0.0945); сред hybrid методите най-добър е **Hybrid+Social** (O = 0.0754 при v4).

5. **Hyperparameter tuning** (v4) подобрява **Hybrid** и **Hybrid+Social** без промяна в данните: NDCG@10 за Hybrid+Social от 0.0649 → 0.0689.

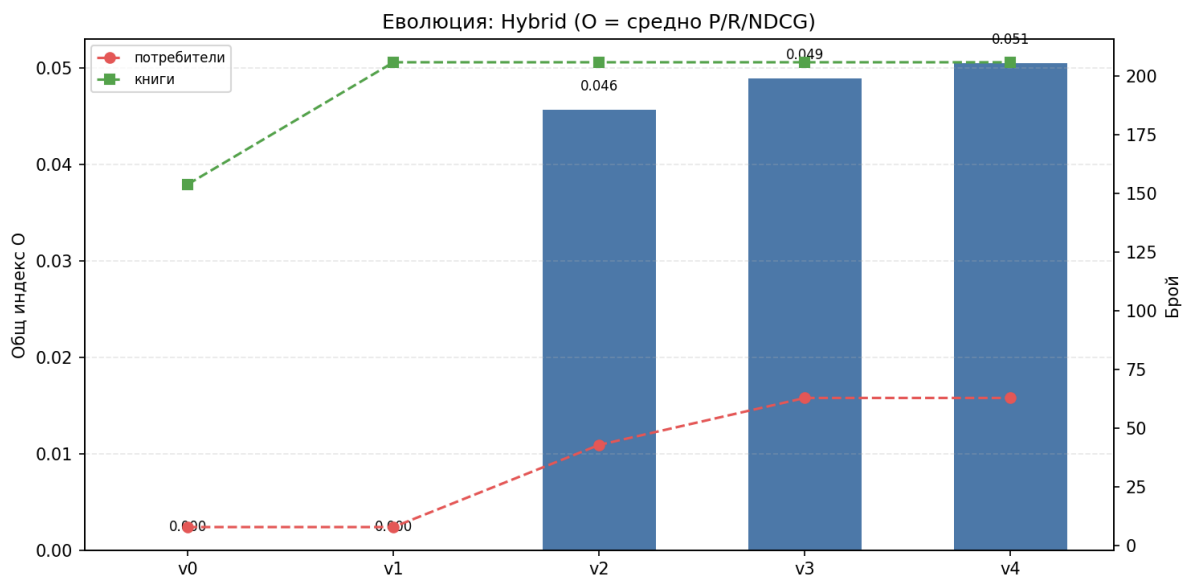
Методологическа бележка: Leave-last-out по последен положителен сигнал за всеки потребител; CF се тренира без отделените взаимодействия; CBF/hybrid не виждат същия ред за съответния потребител.

8. Графики

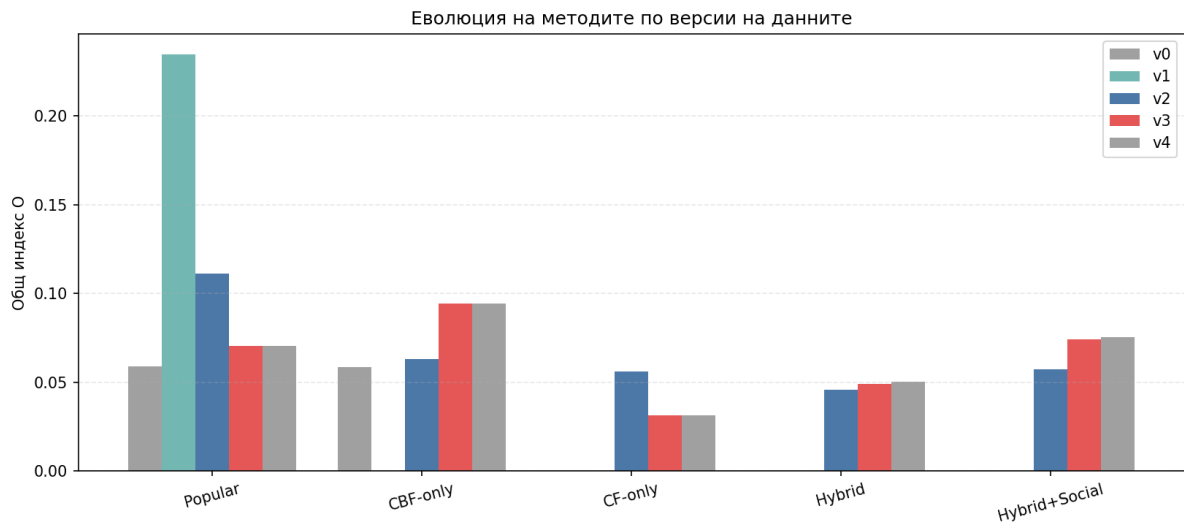
Фиг. 1 — Еволюция Hybrid+Social (v0→v4)



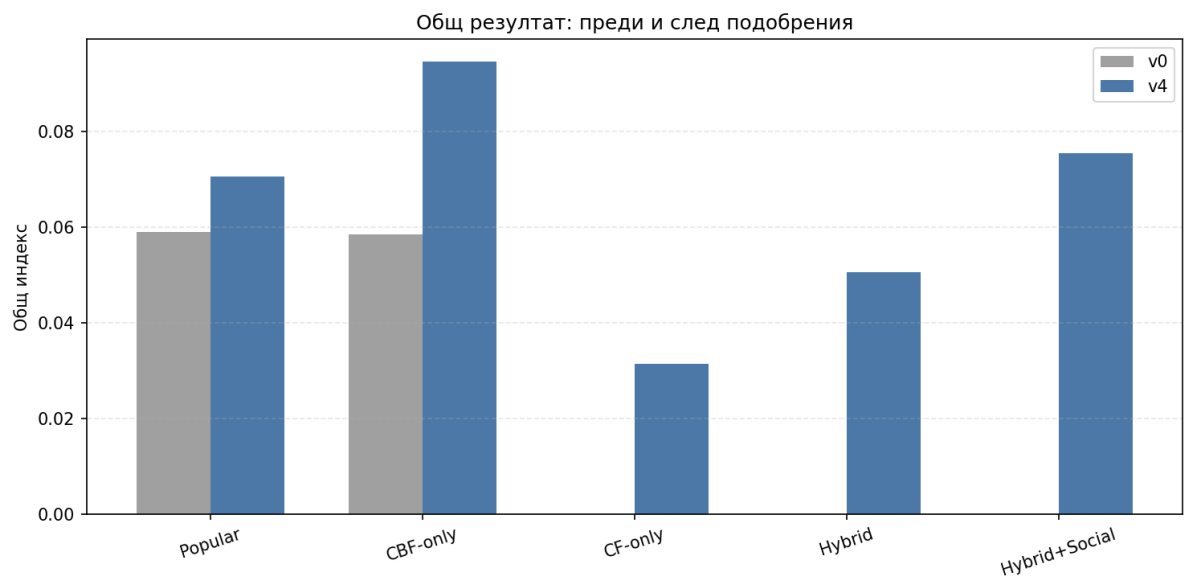
Фиг. 1b — Еволюция Hybrid (v0→v4)



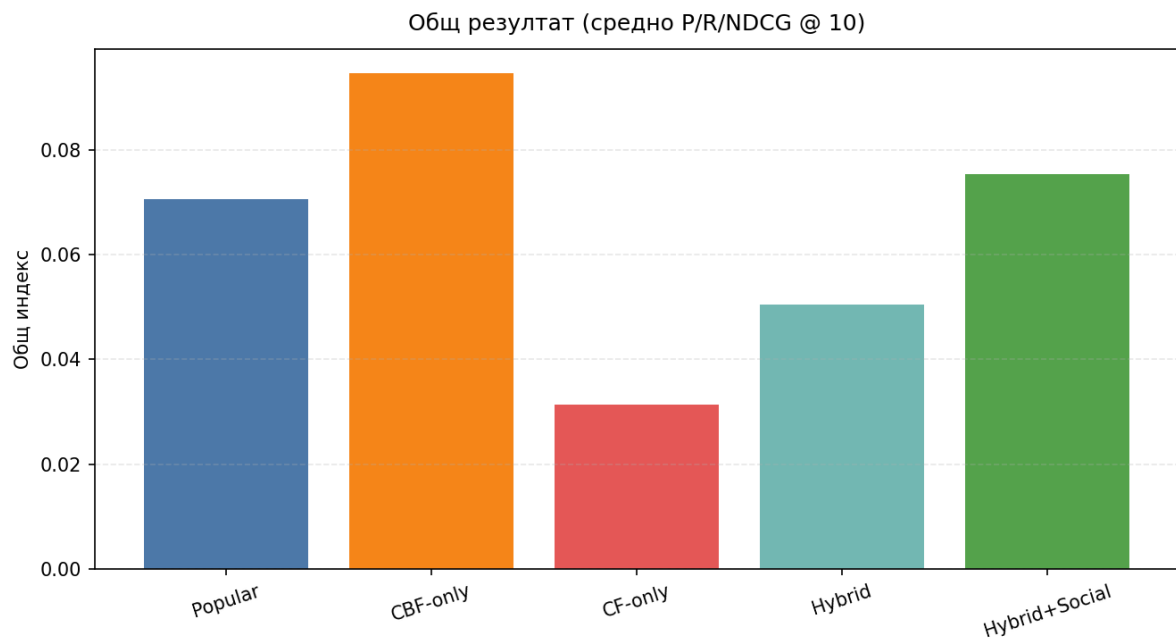
Фиг. 2 — Еволюция на всички методи по версии



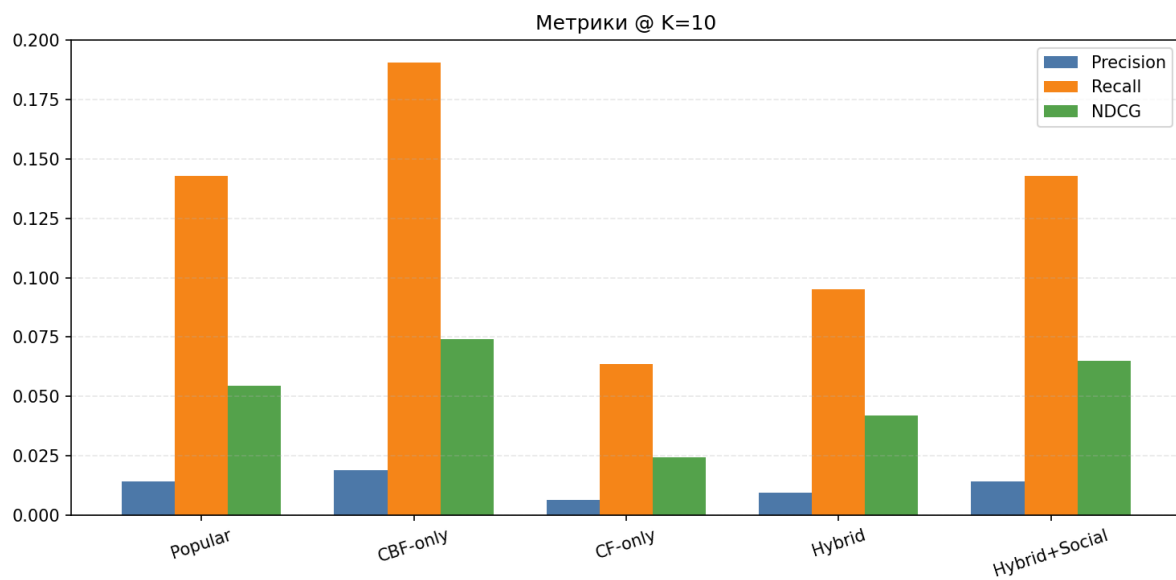
Фиг. 3 — Сравнение v0 vs v4



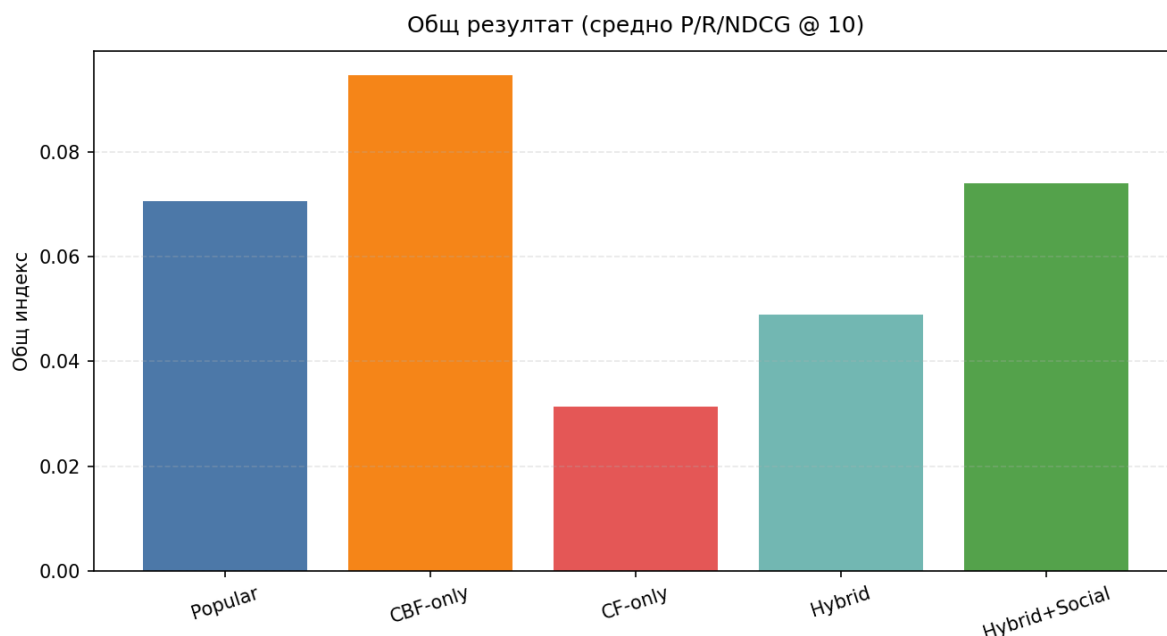
Фиг. 4 — Общ резултат (след tuning, v4)



Фиг. 5 — Precision / Recall / NDCG (v4)



Фиг. 6 — Преди tuning (v3)



9. Извод

1. При малък брой потребители (8 демо акаунта) **Popular** и **CBF-only** са единствените методи с ненулеви offline попадения; **CF** и **Hybrid** не могат да се обучат ефективно заради оскъдна user-item матрица.
2. След разширяване на каталога (Open Library, обогатени тагове), добавяне на синтетични потребители и взаимодействия (до 63 потребителя и 1258 взаимодействия) **CF**, **Hybrid** и **Hybrid+Social** дават измерими резултати. **CBF-only** постига най-висок общ индекс $O = 0.0945$ при финалните данни.
3. **Hybrid+Social** постига по-висок **NDCG@10** от чистия **Hybrid** (0.0689 срещу 0.0469 при v4) — **social signal** допринася при наличие на приятелства в мрежата.
4. **Hybrid** подходът е оправдан теоретично и практически: при **cold start** доминира **CBF**; при повече история **CF** подобрява персонализацията; комбинирането с popularity и social signal дава по-балансиран модел.
5. **Hyperparameter tuning** (оптимизация 3) подобрява **Hybrid** и **Hybrid+Social** без нови данни: NDCG@10 за Hybrid+Social нараства от 0.0649 (v3) на 0.0689 (v4). Това показва, че качеството може да се вдига и чрез настройка на теглата и social blend, не само чрез повече данни.
6. **MMR** подобрява потребителското изживяване чрез разнообразие в Top-K списъка; ефектът му върху offline метриките е по-слаб и зависи от candidate pool и λ .
7. **Ограничения:** каталогът и потребителите са частично синтетични; leave-last-out използва един скрит положителен сигнал на потребител; липсва

online A/B тест. Абсолютните стойности на метриките са ориентировъчни — по-смислено е сравнението между методите и между версиите v0 → v4.

8. **Бъдеща работа:** реални потребителски данни, по-богати протоколи за оценка (k-fold, временни прозорци), online експерименти и евентуално LLM обогатяване на описания с ръчна валидация.

10. Стартиране и възпроизвеждане

Първоначална инсталация

```
cd Book_recommendation_system
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

За генериране на метрики, графики и PDF:

```
pip install -r requirements-dev.txt
```

Стартиране на уеб приложението

```
python -m book_recsys
```

Отвори в браузъра: <http://127.0.0.1:8000>

Демо вход: **nikol** / **demo123**

(Други демо акаунти: gabriela, mira, stefan и др. — парола **demo123**)

Пълен pipeline за доклада (v0 → v4)

Изпълнява базова оценка, оптимизация на каталога, синтетични данни (2 вълни), hyperparameter tuning и PDF:

```
pip install -r requirements-dev.txt
python scripts/run_all_phases.py --reset
```

Само PDF (след като метриките вече са генерирани):

```
python scripts/build_pdf.py
```

Отделни стъпки (по избор)

```
# Метрики и графики за дадена версия
python scripts/export_evaluation.py --tag v0
python scripts/export_evaluation.py --tag v4

# Подобрения на каталога (Open Library + тагове + приятелства)
python scripts/phase_b_improvements.py

# Hyperparameter tuning
python scripts/tune_hybrid.py --k 10

# Импорт на книги от CSV
python -m book_recsys.import_csv

# Импорт от Open Library
python -m book_recsys.fetch_catalog --limit 30 "science fiction"
```